Date: __/__/__ Assignment - 8 Page No.: ____

Problem Statement - Write a C programme to evaluate postfix expression using stack.

Objective -
1) To study stack & its operations
2) To study the importance of expression evaluation.

Theory

Expression evaluation is the process of calculating the value of an arithmetic expression. A stack is the most popular data structure because it follows the last in first out (LIFO) principle, which matches the order in which operators & operands are processed.

Expression evaluation is commonly performed on :
- Postfix
- Prefix

Infix expressions are usually converted to postfix or prefix before evaluation.

Conclusion - Thus, implemented stack operations to evaluate the value of an expression.

Date : __/__/____

Page No.:

FAQs

1) Explain how prefix & postfix expression is evaluated.

Postfix - Scan from left to right. Push operands into the stack. When an operator is found, pop two operands, perform the operation & push the result back.

Prefix - Scan from right to left. Push operands into the stack. When an operator is found, Pop the two operands, perform the operation, push the result back.

2) What is BOP & BIP ?

BOP (Beginning) : Priority of an operator when it is already present inside the stack.

BIP (Incoming) : Priority of an operator when it is being pushed into stack.

3) Give various applications of stack.

- Expression evaluation
- Expression conversion
- Function call management
- Recursion

Date: __/__/____

Handwritten
Creative Memory...

Page No.: ____

Platform -
- Open source programming tool

Pseudo code

1) Initialize an empty stack
2) Scan the expression from left to right
3) If the symbol is an operand:
→ Push it onto the stack
4) If the symbol is an operator:
→ Pop the top element → operand 2
→ Pop the next element → operand 1
→ Perform operand 1 operand 2
→ Push the result onto stack
5) Repeat until the expression ends
6) Pop the final value from the stack
7) Display the result
8) Stop

Time Complexity

- Time complexity - O(n)
- Space complexity - O(n)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int value)
{
    stack[++top] = value;
}

int pop()
{
    return stack[top--];
}

int main()
{
    char exp[200];
    char *token;
    int a, b, result;

    printf("Enter postfix expression: ");
    fgets(exp, sizeof(exp), stdin);

    token = strtok(exp, " \n");
```

```
while (token != NULL)
{
    if (isdigit(token[0]) ||
        (token[0] == '-' && isdigit(token[1])))
    {
        push(atoi(token));
    }
    else
    {
        b = pop();
        a = pop();

        switch (token[0])
        {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;

            case '/':
                if (b == 0)
                {
                    printf("Division by zero error\n");
                    return 1;
                }
                result = a / b;
                break;

            default:
                printf("Invalid operator\n");
                return 1;
```

```
            }

            push(result);
        }

        token = strtok(NULL, " \n");
    }

    if (top == 0)
        printf("Result = %d\n", pop());
    else
        printf("Invalid expression\n");

    return 0;
}
```